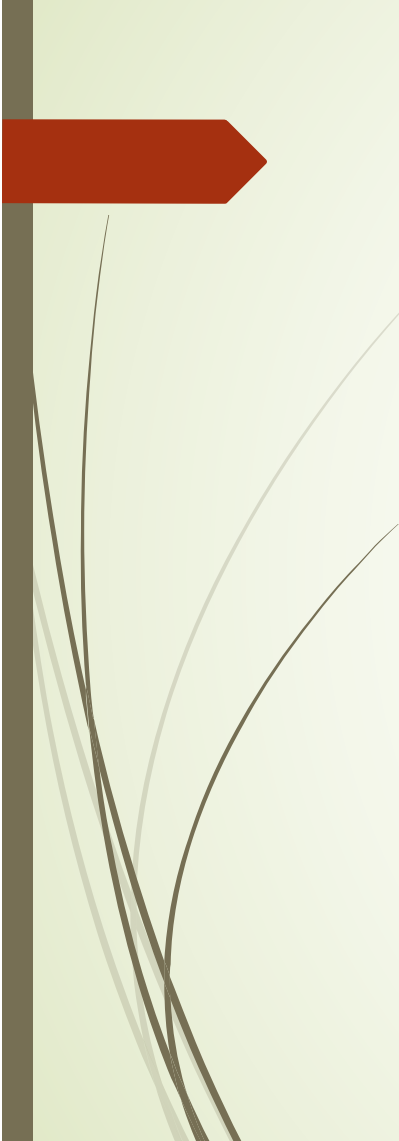




M. Di Ianni

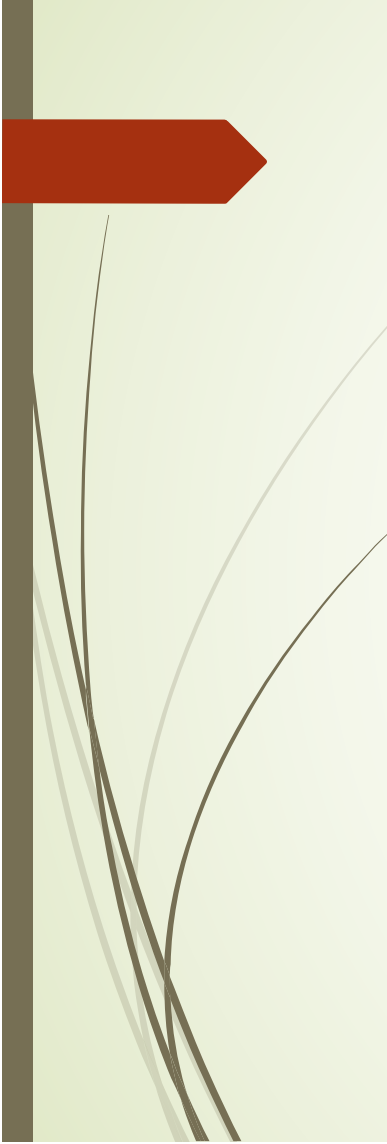
# Lezione 12 – modelli generativi

Lezione del 27/03/2026



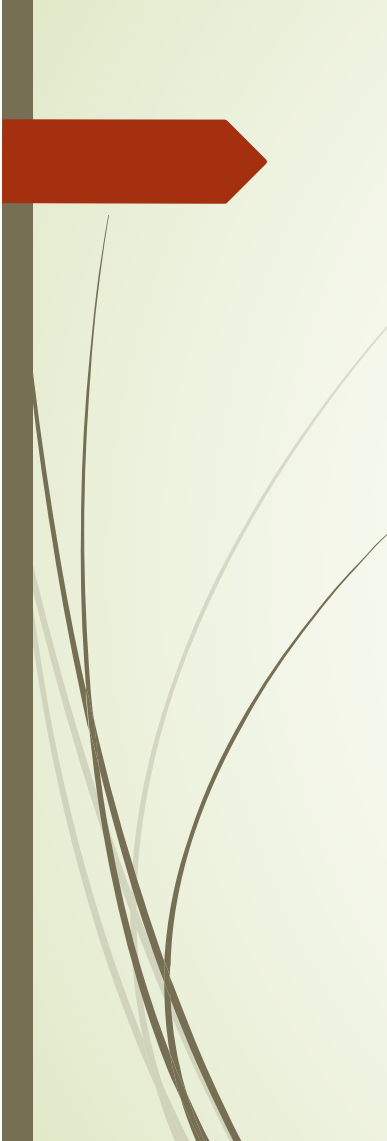
## Ri-facciamo il punto

- Siamo partiti cercando di capire come risolvere automaticamente i problemi
- E abbiamo studiato la soluzione proposta da Alan Turing che, partendo dalla sua analisi del processo di soluzione è arrivato a definire un modello di calcolo: la Macchina di Turing
  - che è un linguaggio per descrivere algoritmi
  - e ogni macchina di Turing è un algoritmo
- Poi, abbiamo introdotto i concetti di linguaggi decidibili e accettabili, e di funzioni calcolabili
  - che corrispondono, informalmente, ai problemi che sappiamo risolvere con la Macchina di Turing
- e abbiamo dimostrato che esistono linguaggi non decidibili o non accettabili e funzioni non calcolabili
  - la possibilità che esistano problemi irrisolvibili – con la Macchina di Turing



# Modelli di calcolo e linguaggi

- Abbiamo già detto che sono stati definiti numerosi modelli di calcolo
  - ragionevoli
- che sono **Turing-equivalenti**
- E fra questi modelli abbiamo visto, a titolo di esempio un linguaggio di programmazione: il PascalMinimo
  - che abbiamo dimostrato essere Turing-equivalente
- Sia la Macchina di Turing che il PascalMinimo si occupano dei linguaggi nella medesima maniera
  - dato un linguaggio (ossia un insieme di parole)
  - progettiamo una macchina di Turing o un programma in PascalMinimo
  - che sappia riconoscere le parole che appartengono a quel linguaggio
- ossia, sono modelli di calcolo *riconoscitori*



# Modelli di calcolo e linguaggi

- Tuttavia, è possibile approcciarsi ai linguaggi in modo diverso
  - in un certo senso, duale
- possiamo progettare **uno strumento capace di generare tutte (e sole) le parole di un linguaggio**
- un siffatto modello di calcolo è un *modello generativo* e prende il nome di *Grammatica*
- e in questa e nelle prossime lezioni studieremo le grammatiche
  - come sono definite
  - in che senso generano linguaggi
  - le proprietà che le caratterizzano
- e le relazioni fra grammatiche e macchine di Turing



# Grammatiche

- Una grammatica è una quadrupla  $G = \langle V_T, V_N, P, S \rangle$  dove
  - $V_T$  è l'insieme dei simboli terminali (che corrisponde all'alfabeto  $\Sigma$ )
  - $V_N$  è l'insieme dei simboli non terminali
  - $S \in V_N$  è un particolare simbolo non terminale detto assioma
  - $P$  è l'insieme delle produzioni, ossia, un insieme di coppie di parole  $(\alpha, \beta) \in (V_T \cup V_N)^* \times (V_T \cup V_N)^*$  tali che  $\alpha$  contiene almeno un simbolo in  $V_N$ 
    - una produzione  $(\alpha, \beta)$  viene indicata nella forma  $\alpha \rightarrow \beta$
    - almeno una produzione è nella forma  $S \rightarrow \beta$
- Obiettivo di una grammatica è generare parole di  $V_T^*$ 
  - a partire dall'assioma
  - e applicando, una dopo l'altra, le produzioni in  $P$
- Prima di capire come si fa a generare una parola mediante una grammatica, abbiamo bisogno di un po' di notazioni

# Grammatiche

- Siano  $\alpha, \beta$  due parole: con  $\alpha\beta$  indichiamo la parola che è la concatenazione di  $\alpha$  e  $\beta$ 
  - ESEMPIO: se  $\alpha = 001$  e  $\beta = abba$ , allora  $\alpha\beta = 001abba$

- Sia  $G = \langle V_T, V_N, P, S \rangle$  una grammatica
- Siano  $x, y \in (V_T \cup V_N)^*$ : scriviamo  $x \Rightarrow_G y$  e diciamo che in  $G$   $y$  **deriva direttamente** da  $x$  se
  - $x = \alpha_1 \alpha \alpha_2$ ,  $y = \alpha_1 \beta \alpha_2$  con  $\alpha_1, \alpha, \alpha_2, \beta \in (V_T \cup V_N)^*$ , e
  - $\alpha \rightarrow \beta$  è una produzione in  $P$

- Siano  $x, y \in (V_T \cup V_N)^*$ : scriviamo  $x \Rightarrow_G^* y$  e diciamo che  $y$  **deriva** (in  $G$ ) da  $x$  se esiste una sequenza di parole  $x_1, x_2, \dots, x_n \in (V_T \cup V_N)^*$  tali che

$$x \Rightarrow_G x_1 \Rightarrow_G x_2 \dots \Rightarrow_G x_n \Rightarrow_G y$$

- Una parola  $y \in V_T^*$  (di soli caratteri terminali) **è generata da  $G$**  se  $S \Rightarrow_G^* y$
- Una parola generata da  $G$  prende il nome di **forma di frase**

# Grammatiche

- Sia  $G = \langle V_T, V_N, P, S \rangle$  una grammatica
- Il **linguaggio generato da  $G$** , che indichiamo con  **$L(G)$** , è l'insieme delle forme di frase generate da  $G$ , ossia

$$L(G) = \{ y \in V_T^* : S \Rightarrow_G^* y \}$$

- Un altro paio di notazioni:
  - Se due o più produzioni hanno la stessa parte sinistra esse si possono raggruppare mediante il simbolo  $|$  (da leggere come "oppure")
    - ad esempio, le due produzioni  $S \rightarrow a$  e  $S \rightarrow b$  possono essere scritte nella forma  $S \rightarrow a \mid b$
  - **$\epsilon$  rappresenta la parola vuota** (ossia, la parola che non contiene alcun carattere)
  - una produzione del tipo  $\alpha \rightarrow \epsilon$  viene chiamata  **$\epsilon$ -produzione**
  - **$\Lambda$  indica il linguaggio vuoto**

## ESEMPIO 1

- Sia  $G = \langle V_T, V_N, P, S \rangle$  la grammatica tale che
  - $V_T = \{ a, b \}$ ,  $V_N = \{ S, A, B \}$
  - $P = \{ S \rightarrow aSB, S \rightarrow aB, B \rightarrow b \}$
- In questo caso, il linguaggio generato da  $G$  è il linguaggio delle parole che iniziano con un certo numero non nullo di caratteri  $a$  e terminano con lo stesso numero di caratteri  $b$
- Vediamolo derivando qualche forma di frase
  - $S \rightarrow aB$  – che contiene una  $a$  e una  $b$
  - $S \rightarrow aSB \Rightarrow_G aSb \Rightarrow_G aabb$
  - $S \rightarrow aSB \Rightarrow_G aaSBB \Rightarrow_G aaSbB \Rightarrow_G aaSBbB \Rightarrow_G aaabBbB \Rightarrow_G aaabBbb \Rightarrow_G aaabbbb$
- Analogamente, per derivare la forma di frase  $aaabbbb$  possiamo:
  - $S \rightarrow aSB \rightarrow aaSBB \rightarrow aaabBB \rightarrow aaabbbB \rightarrow aaabbbb$
  - ma anche  $S \rightarrow aSB \rightarrow aSb \rightarrow aaSBb \rightarrow aaSbb \rightarrow aaabbbb$
  - ossia, una forma di frase può essere derivata in molti modi diversi



## ESEMPIO 1

- Sia  $G = \langle V_T, V_N, P, S \rangle$  la grammatica tale che
  - $V_T = \{ a, b \}, V_N = \{ S, A, B \}$
  - $P = \{ S \rightarrow aSB, S \rightarrow aB, B \rightarrow b \}$
- Per dimostrare che il linguaggio generato da  $G$  è il linguaggio delle parole contenenti lo stesso numero di caratteri  $a$  e  $b$ , ossia,  $L(G) = \{ a^n b^n : n \in \mathbb{N} \}$  occorre dimostrare che
  - 1) comunque si scelga  $n \in \mathbb{N}$ , la parola  $a^n b^n$  può essere derivata da  $S$
  - 2) che ogni parola derivata da  $S$  è nella forma  $a^n b^n$  per qualche  $n \in \mathbb{N}$
- 1) Comunque si scelga  $n \in \mathbb{N}$ , la parola  $a^n b^n$  può essere derivata da  $S$ 
  - comunque si scelga  $n \in \mathbb{N}$ ,
$$S \rightarrow aSB \Rightarrow_G aaSBB \Rightarrow_G aaaSBBB \Rightarrow_G \dots (n-1 \text{ volte}) \Rightarrow_G a^{n-1}SB^{n-1} \Rightarrow_G a^n B^n$$
$$\Rightarrow_G a^n b B^{n-1} \Rightarrow_G a^n bb B^{n-2} \Rightarrow_G \dots \Rightarrow_G a^n b^{n-1} B \Rightarrow_G a^n b^n$$
  - dunque,  $\{ a^n b^n : n \in \mathbb{N} \} \subseteq L(G)$

# ESEMPIO 1

- Sia  $G = \langle V_T, V_N, P, S \rangle$  la grammatica tale che
  - $V_T = \{ a, b \}, V_N = \{ S, B \}$
  - $P = \{ S \rightarrow aSB, S \rightarrow aB, B \rightarrow b \}$
- 2) Ogni parola derivata da  $S$  è nella forma  $a^n b^n$  per qualche  $n \in \mathbb{N}$ , infatti (informalmente):
  - 2.1) osserviamo che le parole nella parte destra delle due produzioni  $S \rightarrow aSB$  e  $S \rightarrow aB$  contengono lo stesso numero di caratteri  $a$  e di caratteri in  $\{b, B\}$
  - 2.2) se inizialmente utilizziamo la produzione  $S \rightarrow aB$ , poiché  $aB$  contiene il non terminale  $B$ , dobbiamo poi utilizzare la produzione  $B \rightarrow b$ : dunque  $S \Rightarrow_G ab$ ;
  - 2.3) se inizialmente utilizziamo la produzione  $S \rightarrow aSB$  otteniamo una parola (contenente anche non terminali) dobbiamo poi applicare una produzione che parte da  $S$  oppure la produzione  $B \rightarrow b$ : in entrambi i casi, otteniamo una parola che contiene lo stesso numero di caratteri  $a$  e di caratteri in  $\{b, B\}$
  - 2.4) ripetiamo il procedimento al punto 2.3) quante volte vogliamo fino a quando la parola risultante non contiene più alcuna  $S$
  - infine, alla parola ottenuta al punto 2.4) applichiamo la produzione  $B \rightarrow b$  per sostituire, una alla volta, tutte le  $B$  con  $b$  e otteniamo una parola che contiene lo stesso numero di caratteri  $a$  e di caratteri in  $b$
  - pertanto,  $L(G) \subseteq \{ a^n b^n : n \in \mathbb{N} \}$

## ESEMPIO 2

- La grammatica  $G = \langle V_T, V_N, P, S \rangle$  tale che
  - $V_T = \{ a, b \}, V_N = \{ S, A \}$
  - $P = \{ S \rightarrow Ab, A \rightarrow Sa \}$
- non genera alcuna parola (di lunghezza finita) in  $V_T$
- pertanto,  $L(G)$  è il linguaggio vuoto:

$$L(G) = \Lambda$$



## ESEMPIO 3

- Ci proponiamo ora di progettare una grammatica  $G = \langle V_T, V_N, P, S \rangle$  tale che  $L(G)$  sia il linguaggio delle parole palindrome nell'alfabeto  $\{ a, b \}$
- La grammatica che serve allo scopo è la seguente:
  - $V_T = \{ a, b \}, V_N = \{ S \}$
  - $P = \{ S \rightarrow aSa, S \rightarrow bSb, S \rightarrow a, S \rightarrow b, S \rightarrow \varepsilon \}$
- Infatti, informalmente:
  - se inizialmente utilizziamo una delle produzioni  $S \rightarrow a, S \rightarrow b, S \rightarrow \varepsilon$  otteniamo le parole  $a$  oppure  $b$  oppure la parola vuota che sono palindrome,
  - se inizialmente utilizziamo una delle produzioni  $S \rightarrow aSa$  o  $S \rightarrow bSb$  otteniamo una parola palindroma nell'alfabeto  $V_T \cup V_N$ , e quindi, induttivamente, segue che le parole generate da questa grammatica sono palindrome
- provare per credere



## ESERCIZI

- 1) Modificare la grammatica dell'ESEMPIO 3 in modo tale che generi solo parole palindrome di lunghezza dispari
- 2) Modificare la grammatica dell'ESEMPIO 3 in modo tale che generi solo parole palindrome di lunghezza pari
- 4) Progettare una grammatica che generi tutte le parole binarie non vuote
  - $S \rightarrow 0S \mid 1S \mid 0 \mid 1$
- 5) Progettare una grammatica che generi l'insieme di parole  $\{ a^n b^m c^h : n, m, h \in \mathbb{N} \}$
- 6) Progettare una grammatica che generi l'insieme di parole  $\{ a^n b^n c^n : n \in \mathbb{N}^+ \}$ 
  - più complesso dei precedenti

## Soluzione dell'esercizio 5)

- 5) Progettare una grammatica che generi il linguaggio  
$$L = \{ a^n b^m c^h : n, m, h \in \mathbb{N} \}$$
- Una grammatica  $G = \langle V_T, V_N, P, S \rangle$  che genera  $L$  è definita dagli insiemi  $V_T = \{a, b, c\}$  e  $V_N = \{S, A, B, C\}$  e dalle produzioni seguenti
  - $S \rightarrow ABC,$
  - $A \rightarrow aA \mid a \mid \varepsilon,$
  - $B \rightarrow bB \mid b \mid \varepsilon,$
  - $C \rightarrow cC \mid c \mid \varepsilon$
- $L$  è generato anche dalla grammatica definita dagli insiemi  $V_T = \{a, b, c\}$  e  $V_N = \{S, B, C\}$  e dalle produzioni seguenti
  - $S \rightarrow aS \mid B,$
  - $B \rightarrow bB \mid C$
  - $C \rightarrow cC \mid \varepsilon$

## Soluzione dell'esercizio 6)

- 6) Progettare una grammatica che generi il linguaggio  
 $L = \{ a^n b^n c^n : n \in \mathbb{N}^+ \}$
- Sia  $G = \langle V_T, V_N, P, S \rangle$  la grammatica definita dagli insiemi  $V_T = \{a, b, c\}$  e  $V_N = \{S, A, B\}$  e dalle produzioni seguenti
  - $S \rightarrow aBSc \mid abc,$
  - $Ba \rightarrow aB,$
  - $Bb \rightarrow bb$
- Dimostriamo ora che  $G$  genera  $L$

## Soluzione dell'esercizio 6)

- 6) Sia  $L = \{ a^n b^n c^n : n \in \mathbb{N}^+ \}$  e  $G = \langle V_T, V_N, P, S \rangle$  la grammatica definita dagli insiemi  $V_T = \{a, b, c\}$  e  $V_N = \{S, A, B\}$  e dalle produzioni seguenti

- $S \rightarrow aBSc \mid abc,$
- $Ba \rightarrow aB,$
- $Bb \rightarrow bb$

- Dimostriamo che **6.1)** le parole di  $L$  sono generate da  $G$  e **6.2)** le parole generate da  $G$  sono in  $L$

- 6.1)** dimostriamo per induzione che le parole di  $L$  sono generate da  $G$ :
  - base dell'induzione ( $n=1$ ): poiché  $G$  contiene la produzione  $S \rightarrow abc$  allora  $G$  genera la parola  $abc$
  - ipotesi induttiva: supponiamo che  $S \Rightarrow_G^* a^i b^i c^i$  per ogni  $i \leq n$
  - dimostriamo che  $G$  genera la parola  $a^{n+1} b^{n+1} c^{n+1}$ :

$$S \rightarrow aBSc \Rightarrow_G^* aB a^n b^n c^n c \Rightarrow_G aaBa^{n-1}b^n c^{n+1} \Rightarrow_G aaaBa^{n-2}b^n c^{n+1} \Rightarrow_G \dots \Rightarrow_G a^{n+1}Bb^n c^{n+1} \Rightarrow_G a^{n+1}b^{n+1}c^{n+1}$$

- dove la derivazione rossa segue dall'ipotesi induttiva



## Soluzione dell'esercizio 6)

- 6) Sia  $L = \{ a^n b^n c^n : n \in \mathbb{N}^+ \}$  e  $G = \langle V_T, V_N, P, S \rangle$  la grammatica definita dagli insiemi  $V_T = \{a, b, c\}$  e  $V_N = \{S, A, B\}$  e dalle produzioni seguenti
  - $S \rightarrow aBSc \mid abc,$
  - $Ba \rightarrow aB,$
  - $Bb \rightarrow bb$
- 6.2) Verifichiamo le parole generate da  $G$  sono parole di  $L$ :
  - Sia  $x \in V_T^*$  una forma di frase generata da  $G$  e sia  $S \Rightarrow_G x_1 \Rightarrow_G x_2 \Rightarrow_G \dots \Rightarrow_G x_n \Rightarrow_G x$  la catena di derivazioni che permette di ottenere  $x$ 
    - si osservi che, poiché la parte destra delle due produzioni che partono dall'assioma  $S$  hanno la  $a$  come primo carattere, ogni  $x_i$  inizia con il carattere  $a$
  - sia  $x_k \in (V_T \cup V_N)^*$  la prima parola nella catena che non contiene  $S$  (ossia,  $k \leq n$  e  $x_1, x_2, \dots, x_{k-1}$  contengono  $S$ )
  - se  $k = 1$ , allora  $x_k = x_1 = abc = x$  che è una parola appartenente a  $L$
  - se  $k > 1$ , allora [pagina successiva]

## Soluzione dell'esercizio 6)

- 6) Sia  $L = \{ a^n b^n c^n : n \in \mathbb{N}^+ \}$  e  $G = \langle V_T, V_N, P, S \rangle$  la grammatica definita dagli insiemi  $V_T = \{a, b, c\}$  e  $V_N = \{S, A, B\}$  e dalle produzioni seguenti
  - $S \rightarrow aBSc \mid abc,$
  - $Ba \rightarrow aB,$
  - $Bb \rightarrow bb$
- **6.2)** Verifichiamo le parole generate da  $G$  sono parole di  $L$ :
  - Sia  $x \in V_T^*$  una forma di frase generata da  $G$  e sia  $S \Rightarrow_G x_1 \Rightarrow_G x_2 \Rightarrow_G \dots \Rightarrow_G x_n \Rightarrow_G x$  la catena di derivazioni che permette di ottenere  $x$  (dove ogni  $x_i$  inizia con  $a$ )
  - sia  $x_k \in (V_T \cup V_N)^*$  la prima parola nella catena che non contiene  $S$
  - se  $k > 1$ , allora  $x_k = ayabcc^k = ayabc^{k+1}$  dove  $y$  è una parola di caratteri in  $\{a, B\}$  che contiene  $k-1$   $a$  e  $k$   $B$  (dunque,  $x_k$  contiene  $k+1$   $a$ ,  $k+1$   $c$ , e  $k+1$  caratteri in  $\{B, b\}$ )
  - allora, l'unico modo per eliminare i non terminali  $B$  da  $y$  è quello di spostare tutte le  $B$  immediatamente a destra del carattere  $b$  più a sinistra (utilizzando la produzione  $Ba \rightarrow aB$ ) e poi utilizzare la produzione  $Bb \rightarrow bb$ : in questo modo si ottiene una parola che inizia con  $k+1$   $a$ , prosegue con  $k+1$   $b$ , e termina con  $k+1$   $c$
  - allora  $x = a^{k+1}b^{k+1}c^{k+1}$  che è una parola appartenente a  $L$